SocraCode: Project Proposal

SocraCode is a web-based tutor with a dashboard, plus a VS Code extension, that guides students, with or without an instructor, to fix their own bugs in Python, Java and C++ through a five-level hint ladder.

Abstract

SocraCode is an AI tutor for beginner programmers learning Python, Java and C++. A shared analysis backend serves two interfaces: a React web portal with a code editor, and a VS Code extension for in-editor hints. Both capture a failing run and give graded hints, from a guiding question up to a full solution that unlocks only on request. Instructor views and a dashboard show which concepts cause trouble. Students can also use it alone, with no instructor or assignment, and still get hints and a personal dashboard.

Problem statement

Mainstream AI coding assistants optimise for speed, so beginners can paste an error and receive a finished fix without understanding it. This helps productivity but weakens learning: the student gets working code and skips the reasoning that builds debugging skill.

Current tools also have no model of the learner. They do not track which concepts a student keeps getting wrong, and they cannot hold back a solution on purpose. Instructors have no way to make an AI assistant follow an assignment's constraints.

SocraCode addresses this gap with three ideas:

- Hints are graded, and each level unlocks only after the previous one.
- The full solution needs a deliberate action from the student.
- Struggles are logged by concept, so the student and an instructor can see patterns.

Objectives

1. Build a React web app with a code editor that runs a student's program (Python, Java or C++) and captures the error, output and code context.
2. Train our own model from scratch, with no pretrained weights and no fine-tuning, to predict the bug category and the buggy line, and combine it with tree-sitter parsing and rules.
3. Support Python first, then Java, then C++, with bug categories shared across languages and rules specific to each.
4. Implement the five-level hint ladder with strict unlocking, using templates so levels 1 to 4 cannot contain the full fix by design.
5. Build instructor views for assignments and a dashboard showing struggles by concept, hint levels used and time to fix.
6. Support independent use: a free-practice mode where a student gets hints and a personal dashboard with no instructor, class or assignment.
7. Build a VS Code extension as a second interface that sends the same code and error context to the shared backend and shows the hint ladder inside the editor.

8. Evaluate the system with a user study and automated checks for diagnosis accuracy and early-level hint leaks.

Proposed solution

The student writes code in the browser editor, runs it, and clicks "Help me" when it fails. The web app asks the backend server to analyse the error and code; the backend picks a bug category and fills the hint template for the current level. Levels unlock in order, and the final level needs a deliberate click.

Level	Name	What the student gets	Example
1	Nudge	A guiding question	"What value should i have on the last loop pass?"
2	Concept	The concept involved	"This looks like an off-by-one error in a loop boundary."
3	Location	The problem area	"Look closely at line 12, the loop condition."
4	Partial solution	A similar example or pseudo-code, not their exact fix	A small correct loop on different data
5	Full solution	Complete fix with explanation, only on explicit request	Corrected code plus why it works

Level 5 comes from the instructor's reference solution (teacher mode) or from a rule-based fix for the supported bug categories. The engine never guesses a fix it cannot verify. Teacher mode also lets an instructor attach constraints to an assignment, so hints stay aligned with the course.

The same flow is available inside VS Code. The extension runs the student's program on their own machine, captures the error and output, and sends only the code and error text to the same backend. The hint ladder appears in a sidebar panel, and each attempt is logged to the same database, so web and extension activity appear together on the dashboard.

SocraCode also works without an instructor. In free-practice mode a student writes or pastes any program, asks for hints and sees a personal history and dashboard; no class, assignment or reference solution is needed. Without a reference solution, level 5 is offered only for bug categories the rule base can fix with certainty. For other bugs the ladder stops at level 4, and SocraCode says so rather than guessing.

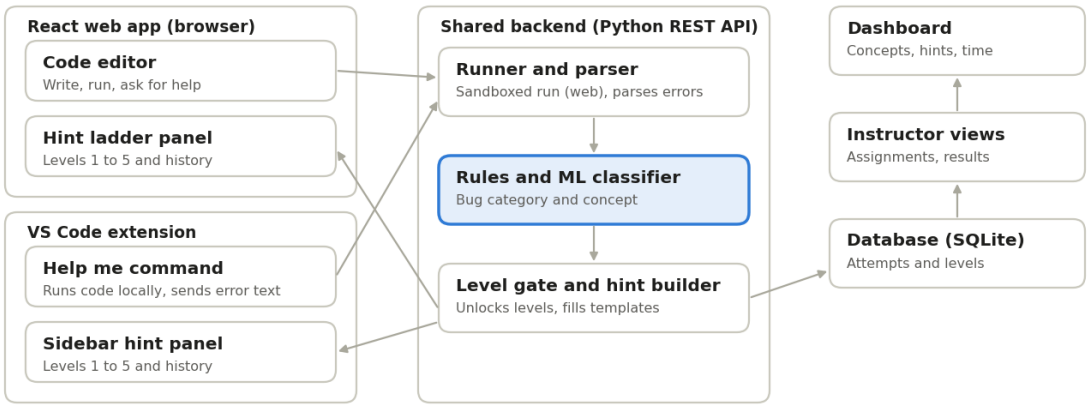
Analysis engine

The engine turns a failing run into a bug category, a likely buggy line and a concept.

Component	Technique	Job
Code runner	Sandboxed server processes: python, javac and java, g++	Compiles and runs the student's program and captures output and

Component	Technique	Job
		errors
Error parser	Parsers for Python tracebacks, javac and g++ messages	Extracts error type, message and line number
Static checks	tree-sitter grammars for Python, Java and C++	Turns code into tokens and syntax trees for the model, and finds simple patterns
Trained model	Small neural network in PyTorch (token-level CNN or BiLSTM), trained from scratch	Predicts the bug category and the likely buggy line, with a confidence score
Baseline model	scikit-learn (logistic regression or random forest) on the same data	A simpler comparison for the evaluation, and a fallback
Rule base	If-then expert-system rules	Handles clear-cut cases such as syntax and compile errors
Example retrieval	TF-IDF similarity over a library of worked examples	Finds a similar example for level 4
Hint templates	One template set per level, bug category and language	Produces hints without a language model
Student model	Counts and recency per concept	Records which concepts a student struggles with

System architecture



A hint request from either interface goes over HTTP to the same Python backend on the project web server. For the web app, the backend runs the code in a sandbox; for the extension, the student's own machine runs the code and sends only the code and error text. The backend parses the failure, builds

the hint for the current level, logs each attempt to a SQLite database, and serves the data the dashboard reads. The level gate is enforced in the backend, so both interfaces follow the same rules

Deployment

SocraCode is deployed as one hosted backend with two interfaces. Students and instructors use the web portal in a browser with nothing to install; students who prefer VS Code can add the extension, which talks to the same server. A student can register and practise alone, or join an instructor's class.

User	How they use it	Network needed
Student (web)	Opens the web app in a browser, writes and runs code, and asks for hints; the code runs on the server in a sandbox	Internet or campus network
Student (VS Code)	Installs the extension, runs code on their own machine and gets hints in a sidebar; only code and error text go to the server	Internet or campus network
Instructor	Opens the same portal to create assignments and view the class dashboard	Internet or campus network

Technology stack

The stack uses standard tools so a small team can build it in 16 weeks. The React app talks to the Python backend over HTTP, and the whole system is hosted on one server.

Layer	Technology	Job in the project
Web front end	React (Vite), Monaco editor	Code editor, Help me button, hint ladder panel, level gate
VS Code extension	TypeScript, VS Code Extension API, webview panel	Help me command, local code run, sidebar hint ladder, calls the backend API
Backend	Python with FastAPI, hosted on the project server	Exposes one REST API used by the web app and the extension; receives code, runs it, calls the engine, serves dashboard data
Code execution	python, javac and java, g++ as sandboxed server processes	Compiles and runs student code (installed once on the server, inside a sandbox; the extension uses the student's own compilers)
Analysis engine	Python, tree-sitter grammars for Python, Java and C++, error parsers	Parses code and compiler or runtime errors

Layer	Technology	Job in the project
Diagnosis	Rule base plus scikit-learn classifier	Bug category and concept, with a confidence score
Hint content	Per-level, per-language templates and worked examples	Hints without a language model
Storage	SQLite: one server database for all students and instructors	Attempts, hint levels used, concept tags
Dashboard	React with Chart.js or Recharts, bundled locally	Struggles by concept, hints used, time to fix
Testing	pytest, Vitest	Engine and front-end tests

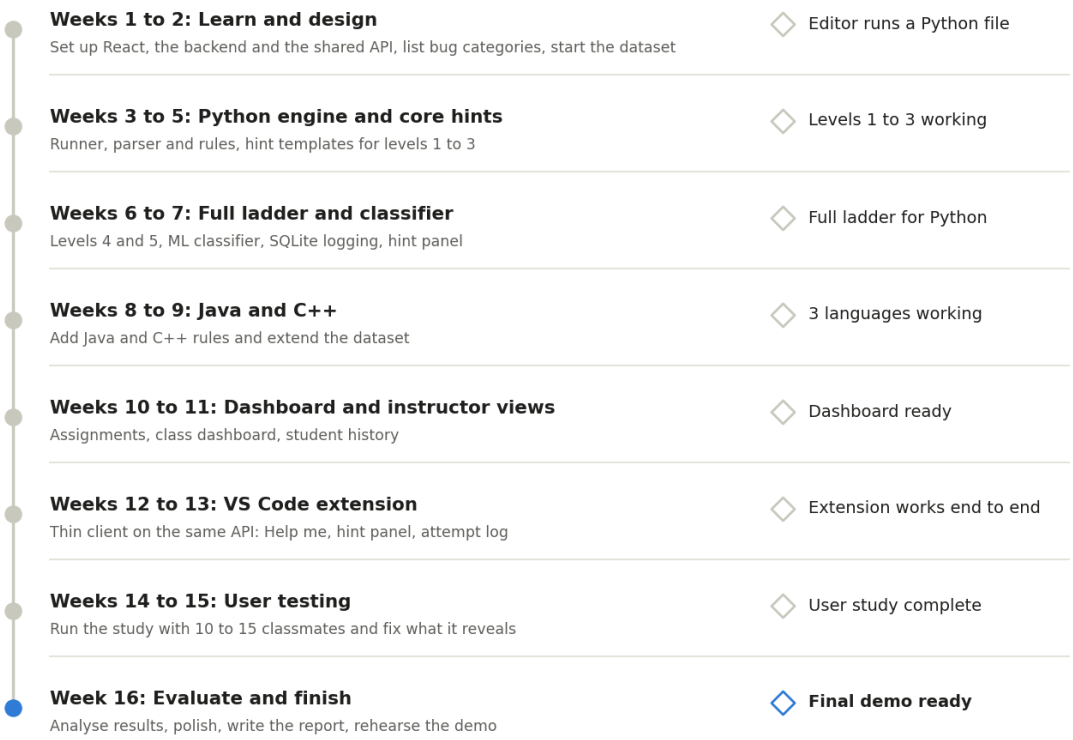
Scope

The build covers Python, Java and C++ in that order, with Python in the most depth and C++ hints driven mainly by compiler errors. The VS Code extension is a thin client on the same backend.

In scope	Out of scope
Error and test-failure capture for Python, then Java, then C++	Languages beyond Python, Java and C++
Analysis engine: rules plus an ML classifier for about 8 shared bug categories	Any external AI service or API
Five-level hint ladder with strict unlocking	Training or fine-tuning large models
React web app with code editor and hint history	Automatic grading of assignments
VS Code extension: Help me command, hint ladder panel and attempt logging through the shared backend API	A separate dashboard inside the extension, and other editors such as JetBrains
Independent use: free-practice mode and a personal dashboard for students with no instructor, class or assignment	Level 5 fixes for bugs the rule base cannot verify
Instructor views and a class dashboard in the hosted web portal	Large-scale public deployment (the demo runs on one lab or personal server)
Teacher mode (optional): reference solution used for level 5	LMS integration and multi-institution accounts
C++ hints driven mainly by compiler errors	Deep static analysis for C++
Student code runs on the server inside a sandbox	Running code outside the sandbox or without

In scope	Out of scope
with time and memory limits	resource limits

Implementation plan



Evaluation plan

Success is measured with a small user study and two automated checks, so the report rests on evidence rather than a demo alone.

Measure	Method	Target
Time to fix	10 to 15 classmates fix the same buggy programs (Python, Java and C++), half with SocraCode and half with their usual tools	Compare group means; report the difference
Understanding	Short follow-up quiz on each bug, plus student self-rating	Higher or equal score than the control group
Diagnosis accuracy	Run a held-out set of labelled buggy programs per language through the engine	Report accuracy per bug category and language, and a confusion matrix

Measure	Method	Target
Early-level leak check	Compare level 1 to 4 outputs with the stored fix for every test program	Zero leaks, checked by an automated test
Hint relevance	Student rates each hint as helpful or not	Report the percentage rated helpful
Usability	A short questionnaire after the session	Report the average score and written feedback
Interface choice	Participants may use the web portal or the VS Code extension; compare hints used and ratings by interface	Report usage and preference for each interface

Risks and mitigations

The biggest risks are misdiagnosed bugs and hints that feel generic; the rest are scope and time risks.

Risk	Likelihood	Mitigation
Engine misdiagnoses the bug	High	Start with about 8 common bug categories, show a confidence score, and fall back to a generic level 1 question when confidence is low
Hints feel generic without a language model	High	Invest in good templates per category, let students mark a hint "not helpful", and state the limit honestly in the report
Supporting three languages multiplies rules and dataset work	High	Finish Python fully first, then Java; give C++ mostly compiler-error hints; share bug categories across languages
Building the labelled dataset takes too long	Medium	Target 100 to 150 Python programs, create variants by inserting known bugs into correct code, and start in week 1
Learning React and the backend slows early weeks	Medium	Get "editor runs a Python file" working in weeks 1 and 2, start from the Vite template, and keep the UI to a few screens
Running student code safely on the server	Medium	Run code in Docker with time, memory and network limits, install the JDK and g++ once on the server, and test with deliberately harmful programs such as infinite loops and heavy memory use
Scope creep across engine, languages and dashboard	High	Freeze scope at the end of week 8 and move extras to a stretch list

Risk	Likelihood	Mitigation
Too few classmates for the study	Medium	Recruit in week 12, run short 20-minute sessions, and report results as indicative
VS Code extension takes longer than planned or behaves differently across machines	Medium	Build it last as a thin client on a stable API, limit it to Help me, hint panel and logging, document the required compilers, and drop it to a stretch goal if the web app slips
Free practice has no reference solution, so level 5 is limited	Medium	Offer level 5 only for rule-verified bug categories, stop at level 4 with a clear message otherwise, and report how often this happens

Expected deliverables and conclusion

By the end of week 16 the project delivers:

- A working React web app with a code editor and the five-level hint ladder
- A VS Code extension that shows the same five-level hint ladder inside the editor
- A free-practice mode so students can use SocraCode without an instructor
- An analysis engine (rules, ML classifier, tree-sitter parsing) supporting Python, Java and C++
- The labelled dataset of buggy programs
- Instructor views for assignments and a dashboard
- A source repository with a README and setup guide
- An evaluation report with the user-study results
- A live demo: a buggy program taken through every hint level

SocraCode shifts coding help from giving answers to building understanding, and it does so as a hosted web server. Deep enough to show real engineering (web development, static analysis, rule-based and machine-learning diagnosis, a web portal and an editor extension), and measurable through a direct user study.

User story: how SocraCode is used

The teacher sets the work and reads the results; the student does the debugging, and SocraCode guides without solving it for them. The names below are illustrative.

Role	What they do	What they see
Student	Writes and runs code in the browser editor, asks for hints, and climbs the ladder one level at a time	Their own hint history and weak concepts in the portal
Independent student	Uses free practice with no class or assignment, asks for hints, and climbs the ladder	Own hint history and weak concepts; no instructor view
Instructor	Creates assignments with a reference solution and constraints, and reviews class results	A class dashboard: concepts, hint levels used and time to fix per

Role	What they do	What they see
		assignment
SocraCode	Diagnoses the failure, builds the hint for the current level, and logs the attempt	Nothing; it works in the background

1. **Setup.** Ms. Sara, the instructor, opens the portal and creates the assignment "Sum of a list". She adds a reference solution and a constraint that the answer must use a loop.
2. **The bug.** Ali, a student, opens the assignment in the browser editor and writes a loop that starts at index 1, so his function returns the wrong total and the test fails.
3. **Level 1, nudge.** Ali clicks "Help me". SocraCode asks: "Which items does your loop visit, and does it include the first one?"
4. **Level 2, concept.** Still stuck, Ali asks for the next level: "This looks like an off-by-one error in a loop boundary."
5. **Level 3, location.** Next: "Look closely at line 4, where the loop starts."
6. **Level 4, partial solution.** SocraCode shows a similar loop on different data. Ali spots the difference and fixes his own code. He never needs level 5.
7. **Logging.** The attempt is saved with the bug category (loop boundary), the highest level used (4) and the time to fix.
8. **Student view.** In the portal, Ali sees that loop boundaries are a concept he keeps needing hints for, so he knows what to practise.
9. **Instructor view.** Ms. Sara's dashboard shows that many students needed level 3 or higher on loop boundaries. She spends part of the next lecture on loop ranges.
10. **Full solution.** A student who gives up can click "Show full solution" and confirm. The fix is shown with an explanation, and the dashboard records it so the instructor can follow up with that student.

Steps 7 to 10 use one portal: each student sees only their own history, and the instructor sees the whole class. Students who use the VS Code extension get the same hints inside the editor, and their attempts reach the same dashboard.

Without an instructor, steps 1 and 9 do not apply. Ali opens free practice, writes any program, climbs the ladder in the same way and sees his own history in steps 7 and 8. Level 5 appears only for bug categories the engine can fix with certainty.

Technology behind each step

The same story, showing which part of the tech stack does the work at each point.

Step	What happens	Technology
1. Setup	The instructor creates the assignment and uploads the reference solution	React form, FastAPI, SQLite
2. The bug	The editor runs the code and the backend captures the failing run and test output	React with Monaco editor, FastAPI, python, javac and g++ as sandboxed server processes

Step	What happens	Technology
3. Help me	The app sends the code and error to the engine	HTTP request to the server, Python engine
4. Diagnosis	The engine parses the failure and picks the bug category (loop boundary)	Error parsers, tree-sitter, rule base, scikit-learn classifier
5. Hint building	The template for the current level is filled in, and the level gate blocks skipping ahead	Per-level, per-language hint templates, level gate in the React app
6. Showing hints	Hints and history appear in the hint panel	React components
7. Logging	The attempt, hint level and time are saved	SQLite through Python's sqlite3 module
8 to 9. Dashboard	Student and instructor views draw the charts	React with Chart.js or Recharts, data served by FastAPI
10. Full solution	The fix is shown from the reference solution, with an explanation	Teacher-mode reference solution, text diff
Hosting	The whole system runs on one server that users reach in a browser	FastAPI serving the React build, SQLite
VS Code extension	A student asks for the same hints from inside VS Code	TypeScript extension, same FastAPI endpoints
Free practice	A student practises with no class or assignment and gets the same hint ladder	Same engine and FastAPI endpoints, no assignment record